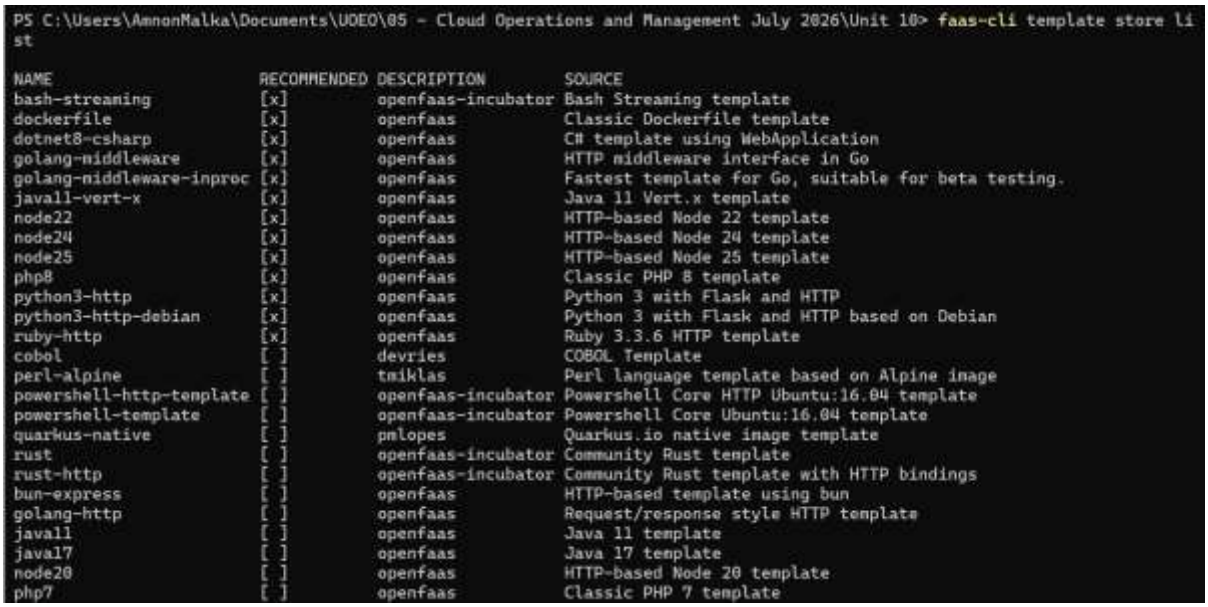
Task ‘Unit 10: Advanced Cloud Technologies (Serverless Computing) Unit 10

Discussion Forum: OpenFaaS - OpenFaaS Serverless Function

OpenFaaS (Functions as a Service) is an open-source serverless platform that enables applications to run as containerized functions. For this project, a simple Python greeting function was developed using the official OpenFaaS python3-http template and packaged into a Docker image. The python3-http template provides an HTTP-based Python environment for OpenFaaS functions.

OpenFaaS (2026a)

Figure 1: OpenFaaS python3-http template list



```
PS C:\Users\AmnonMalka\Documents\UOEO\05 - Cloud Operations and Management July 2026\Unit 10> faas-cli template store li
st

NAME                    RECOMMENDED DESCRIPTION        SOURCE
bash-streaming          [x]         openfaas-incubator Bash Streaming template
dockerfile              [x]         openfaas           Classic Dockerfile template
dotnet8-csharp          [x]         openfaas           C# template using WebApplication
golang-middleware       [x]         openfaas           HTTP middleware interface in Go
golang-middleware-inproc [x]        openfaas           Fastest template for Go, suitable for beta testing.
java11-vert-x           [x]         openfaas           Java 11 Vert.x template
node22                  [x]         openfaas           HTTP-based Node 22 template
node24                  [x]         openfaas           HTTP-based Node 24 template
node25                  [x]         openfaas           HTTP-based Node 25 template
php8                    [x]         openfaas           Classic PHP 8 template
python3-http            [x]         openfaas           Python 3 with Flask and HTTP
python3-http-debian     [x]         openfaas           Python 3 with Flask and HTTP based on Debian
ruby-http               [x]         openfaas           Ruby 3.3.6 HTTP template
cobol                   [ ]         devries            COBOL Template
perl-alpine             [ ]         tmiklas            Perl language template based on Alpine image
powershell-http-template [ ]        openfaas-incubator Powershell Core HTTP Ubuntu:16.04 template
powershell-template     [ ]         openfaas-incubator Powershell Core Ubuntu:16.04 template
quarkus-native          [ ]         pmlopes            Quarkus.io native image template
rust                    [ ]         openfaas-incubator Community Rust template
rust-http               [ ]         openfaas-incubator Community Rust template with HTTP bindings
bun-express             [ ]         openfaas           HTTP-based template using bun
golang-http             [ ]         openfaas           Request/response style HTTP template
java11                  [ ]         openfaas           Java 11 template
java17                  [ ]         openfaas           Java 17 template
node20                  [ ]         openfaas           HTTP-based Node 20 template
php7                    [ ]         openfaas           Classic PHP 7 template
```

Function Development

The function was created using the OpenFaaS Python HTTP template. The function accepts a JSON request containing a name and returns a greeting:

```Python
```

```
import json
def handle(event, context):
    data = json.loads(event.body)
    name = data.get("name", "World")

    return {
        "statusCode": 200,
        "body": json.dumps({"message": f"Hello, {name}!"}),
        "headers": {"Content-Type": "application/json"}
    }
```

The deployment configuration was defined in stack.yaml:

```yaml
version: 1.0

provider:
  name: openfaas
  gateway: http://127.0.0.1:8080

functions:
  greeting:
    lang: python3-http
```

```
    handler: ./greeting
    image: phonixx/openfaas-greeting:latest


configuration:
  templates:
    - name: python3-http
      source: https://github.com/openfaas/python-flask-template
```

The configuration templates section allows the OpenFaaS CLI to retrieve the required template during the build process.

OpenFaaS (2026b)

Containerization and Deployment

The official python3-http template was downloaded and its supplied Dockerfile was used to package the function. A requirements.txt file was also included in the function directory as required by the Python template. During the build process, a Windows-specific issue occurred with faas-cli 0.18.13 when parsing the local template path. Therefore, the Dockerfile supplied by the official OpenFaaS template was used directly with Docker.

The Docker image was successfully built using:

```
docker build -t phonixx/openfaas-greeting:latest .\docker-build
```

The Docker build process was completed successfully and all build stages were completed. The image produced was then labeled as phonixx/openfaas-greeting:latest.

OpenFaaS (2026c)

Figure 2: Docker build process

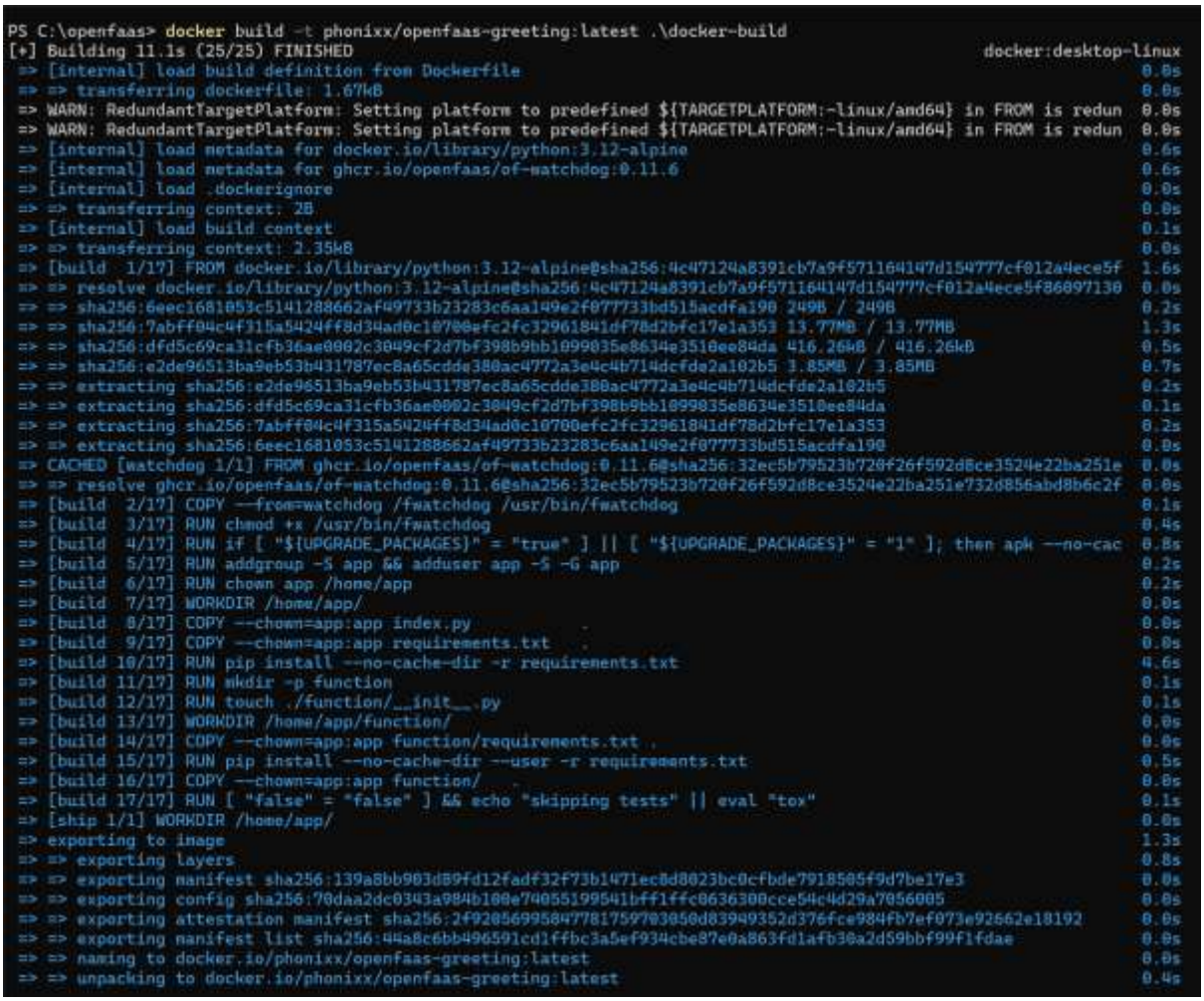


Figure 3: Successful Docker Image deployment



Figure 4: Docker image listing



Figure 5: Running container / function test



(Microsoft Learn, 2026).

Serverless Benefits

OpenFaaS reduces infrastructure management by handling function execution and deployment through its platform. Containerization provides a consistent and portable runtime, while functions can be independently scaled according to demand. Yet, such practice could present some security implications, for instance; vulnerable container images, insecure function dependencies, inadequate API authentication and authorization, or the exposure of sensitive data. The distributed and dynamically scaled nature of serverless functions can also cause security monitoring and incident analysis to be more challenging. Overall, serverless architecture can help to improve deployment flexibility, resource efficiency and operational automation, while also presenting potential performance, cost and management risks that must be taken into consideration.

(Morabito et al., 2023)

References

- Morabito, G., Sicari, C., Ruggeri, A., Celesti, A. and Carnevale, L., 2023. Secure-by-design serverless workflows on the edge–cloud continuum through the osmotic computing paradigm. *Internet of Things*, *22*, p.100737

- Microsoft Learn (2026). Microsoft PowerShell Utility Invoke-RestMethod. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/invoke-restmethod?view=powershell-7.6 [Accessed: 26 September 2026].

- OpenFaaS (2026a) Python. Available at: https://docs.openfaas.com/languages/python/ [Accessed: 26 September 2026].

- OpenFaaS (2026b) Create new functions and templates. Available at: https://docs.openfaas.com/cli/templates/ [Accessed: 26 September 2026].

- OpenFaaS (2026c) Build functions. Available at: https://docs.openfaas.com/cli/build/ [Accessed: 26 September 2026].

- OpenFaaS (2026c) Troubleshooting. Available at: https://docs.openfaas.com/deployment/troubleshooting/ [Accessed: 29 September 2026].

Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. All academic writing, analysis, argumentation, and conclusions represent my original work. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.